

PIXEL SOFTWARE DESIGN

Département de Recherche & Développement

Intelligence Artificielle Avancée

& Federated Learning

- Fondations de l'intelligence artificielle
- Algorithmes d'apprentissage fédéré
- Confidentialité, attaques et défenses
- Applications et grands modèles fédérés
- Framework FedX de référence
- Nouveaux fronts de recherche
- Travaux pratiques, projets et évaluation

Auteur	**Ghiloufi Jamel**
Fonction	**Architecte Logiciel Principal**
Direction	**Département de Recherche & Développement**
Service	**R&D - Intelligence Artificielle Avancée & Apprentissage Fédéré**
Niveau	**Recherche**
Édition	**Édition 2026 – 1.0**
Copyright	**© Pixel Software Design**

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Copyright et mentions légales

Éditeur	Pixel Software Design
Auteur	Ghiloufi Jamel
Fonction	Architecte Logiciel Principal
Département	Département de Recherche & Développement
Intitulé	Intelligence Artificielle Avancée & Federated Learning
Sous-titre	Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Édition	Édition 2026 (version 1.0)
Langue	Français
Niveau	Recherche
Collection	Supports de formation et de recherche, Pixel Software Design

Droits d'auteur

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Droits de l'auteur. Ghiloufi Jamel est l'auteur unique de l'ensemble du contenu scientifique, pédagogique et technique de cet ouvrage. Toute adaptation, traduction, reproduction ou exploitation, y compris partielle, requiert une autorisation écrite préalable.

Droits des tiers. Les algorithmes, modèles et travaux cités restent la propriété de leurs auteurs respectifs. Les références employées figurent en fin d'ouvrage, ce qui permet de remonter au travail original pour toute affirmation chiffrée.

Marques et licences externes. PyTorch, TensorFlow, Flower, Opacus, MONAI, scikit-learn et PySyft sont des logiciels libres sous licences respectives (BSD, Apache 2.0, MIT). Leur usage dans ce document est purement pédagogique et n'emporte aucune prescription commerciale.

Mentions relatives aux données et aux expérimentations

Document pédagogique à usage de formation interne. Les exemples de code fournis sont livrés à titre pédagogique ; les résultats numériques illustrés sont issus d'expérimentations conduites dans un cadre de recherche et ne constituent pas des performances de production.

Ce support synthétise les travaux de recherche du Département de Recherche & Développement de Pixel Software Design sur l'intelligence artificielle distribuée, l'apprentissage fédéré, la protection de la vie privée et les systèmes embarqués intelligents.

Sommaire

Structure détaillée du cours

Partie 1 — IA et Recherche Opérationnelle 8

1.1 Recherche opérationnelle et apprentissage automatique : la couture 9

1.2 Prévision, décision et optimisation : la chaîne complète 10

1.3 Apprentissage par renforcement pour la décision séquentielle 12

1.4 Logistique et tournées de livraison 14

1.5 Ordonnancement et pilotage d'atelier 18

1.6 Énergie, finance et déploiement en production 20

Annexes 30

A.1 Liste des abréviations 31

A.2 Table de notation 31

Partie	Thème	Contenu	Compétence visée	Durée
Partie 1	Fondations	Statistique, optimisation, courbes d'apprentissage	Comprendre et formaliser	2-3 semaines
Partie 2	Algorithmes FL	FedAvg, convergence, hétérogénéité, système	Implémenter et régler	3-4 semaines
Partie 3	Confidentialité	DP, attaques, défenses, agrégation sécurisée	Auditer et sécuriser	2-3 semaines
Partie 4	Applications	FedLLM, santé, V2X, FedRL, TinyML, graphes	Transposer à un domaine	3-4 semaines
Partie 5	Framework FedX	Architecture, API, benchmarks, exploitation	Industrialiser	2 semaines
Partie 6	Nouveaux fronts	FedCL, FedCI, SNN, FedGL, feuille de route	Ouvrir la recherche	2 semaines
Partie 7	TP et projets	9 labs, 5 projets, corrigés, évaluation	Faire et rendre	4-6 semaines

Préface

Comment utiliser ce cours

Ce support est conçu pour un parcours de formation et de recherche couvrant l'ensemble de la chaîne de valeur de l'intelligence artificielle distribuée : les fondations mathématiques, les algorithmes d'apprentissage fédéré, les menaces et défenses, les applications de pointe, une plateforme de référence opérationnelle, et enfin les nouveaux fronts ouverts par la recherche du Département.

Ce n'est pas un manuel de référence à lire d'un trait : c'est un **manuel opératoire**. Chaque chapitre alterne exposition, formalisation, mise en œuvre numérique et évaluation. Les listings sont complets et exécutables ; les tableaux comparatifs servent de grille de décision ; les schémas d'architecture servent de support de revue de conception.

Public et prérequis

Le contenu s'adresse à des étudiants de master et de doctorat, à des ingénieurs IA et à des praticiens de la recherche appliquée. Les prérequis sont la connaissance de l'apprentissage automatique classique, de l'algorithmique numérique et des bases de probabilités. Les parties 1 à 3 sont auto-suffisantes pour un étudiant qui découvre le domaine ; les parties 4 à 7 supposent la mécanique et les outils présentés dans la partie 2.

Conventions de lecture. Une expression mathématique est encadrée par deux signes dollars ; un identifiant, un fichier ou une commande est écrit en caractères à chasse fixe ; les encadrés *Définition*, *Théorème*, *Méthode* et *Attention* signalent les points à maîtriser et les pièges fréquents. Les renvois de la forme « voir chapitre 3.2 » désignent le second chapitre de la troisième partie.

Progression pédagogique

Optimiser sous incertitude, et savoir quand s'arrêter

Partie 1

IA et Recherche Opérationnelle

La recherche opérationnelle optimise une décision sous contraintes déclarées. L'apprentissage automatique estime les grandeurs dont cette décision dépend. Les deux compétences sont complémentaires, et leur couture est exactement l'endroit où se perd la valeur : une erreur d'estimation ne se signale jamais comme telle, elle devient une décision sous-optimale. Ce cours décrit la chaîne complète, du modèle de prévision au solveur, et s'attarde sur les quatre manières d'hybrider — par ordre croissant de prise de risque — ainsi que sur les quatre domaines où la synergie produit un gain mesurable : logistique, ordonnancement, énergie et finance.

Objectifs de la partie

- Formuler la question qui décide de l'hybridation : quelles grandeurs sont incertaines, et quel est le coût d'une erreur d'estimation.
- Distinguer erreur de prévision et erreur d'optimisation, et mesurer la seconde en unités de valeur économique.
- Expliquer pourquoi un simulateur entraîne une politique pour le simulateur, et comment_on rend ce transfert défendable.
- Résoudre un problème de tournées avec fenêtres horaires, et expliquer pourquoi il n'a pas de solution exacte à l'échelle.
- Modéliser un atelier comme un graphe de contraintes, et expliquer le rôle exact du réseau de neurones dans cette modélisation.
- Choisir, parmi quatre modes d'hybridation, celui que l'on peut défendre devant un auditeur de risque.

Plan de la partie

1. Recherche opérationnelle et apprentissage automatique : la couture
2. Prévision, décision et optimisation : la chaîne complète
3. Apprentissage par renforcement pour la décision séquentielle
4. Logistique et tournées de livraison
5. Ordonnancement et pilotage d'atelier
6. Énergie, finance et déploiement en production

Chapitre 1.1

Recherche opérationnelle et apprentissage automatique : la couture

Deux compétences, une seule couture. Tout le reste en découle.

Un modèle d'optimisation et un modèle d'apprentissage répondent à deux questions différentes. Le premier demande : parmi toutes les façons de faire, laquelle est la meilleure sous les contraintes que je déclare ? Le second demande : qu'est-ce qui va réellement se produire, et avec quelle incertitude ? La plupart des systèmes industriels échouent non pas parce que l'un des deux est mauvais, mais parce qu'on a traité la réponse du second comme un nombre certain au lieu d'une loi de probabilité.

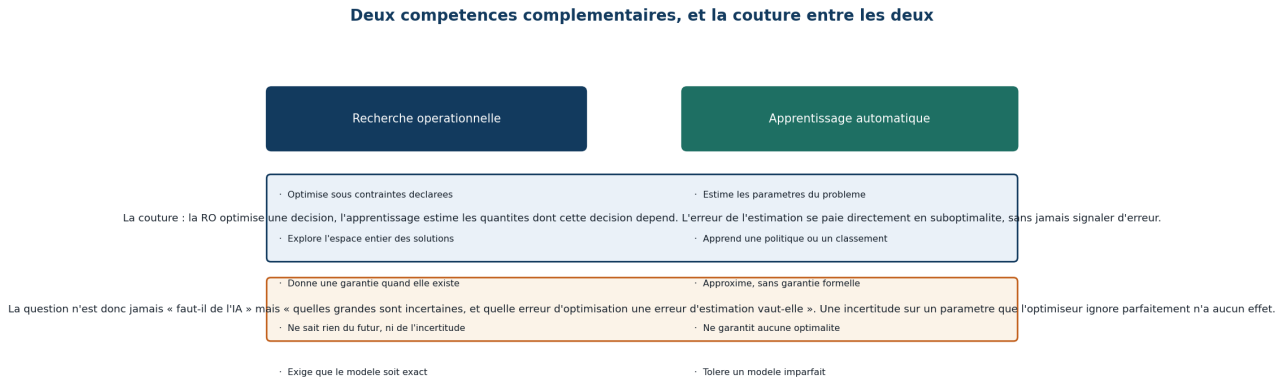


Figure 1 – Deux compétences complémentaires, et la couture entre les deux

Question	Recherche opérationnelle	Apprentissage automatique
Quelle est la meilleure décision ?	répond exactement, si le modèle est juste	ne répond pas : il ne cherche pas d'optimum
Que va-t-il se passer ?	ne répond pas : ses paramètres sont déclarés	répond par une loi estimée, jamais par un nombre
Combien de solutions existe-t-il ?	les énumère quand l'espace le permet	approxime une politique, sans les voir toutes
Que se passe-t-il si le modèle est faux ?	produit un optimum du mauvais problème	produit une décision plausible du mauvais monde
Comment garantir le résultat ?	garantit l'optimalité sous hypothèses	ne garantit rien, et le dit
Comment détecter son erreur ?	ne le peut pas, faute de référence	mesure sa propre incertitude, quand on le lui demande

Tableau 1 – Ce que chacune des deux disciplines sait faire et ignorant

Les deux dernières lignes sont les plus importantes. Une garantie d'optimalité porte sur un modèle, jamais sur le monde ; et une incertitude non demandée n'est jamais fournie.

$$\min_{x \in \square 1 \square} c^{\top} x \text{ sous } A\,x \le b,\ x \in Z^{n}$$

$$c \leftarrow \hat{c},\ E\le ft[\, c(x^{\star}) - c^{\star} \,] \le E\le ft[\, \le ft|\, \hat{c} - c \,| \,] \cdot \le ft|\, x^{\star} - x^{\star}_{\hat{c}} \,| \qquad \text{(R.1)}$$

La première expression est le programme linéaire. La seconde est la question centrale de tout système hybride : l'écart d'optimisation se borne par le produit de l'erreur d'estimation et de la sensibilité de la solution à cette erreur. Un paramètre mal estimé dont la solution y est insensible ne coûte rien ; un paramètre critique, même bien estimé en moyenne, coûte cher dès qu'il est mal estimé.

Figure indisponible

La figure `ro_explosion` n'est pas encore implementee.

Poser la question qui décide de l'hybridation, en trois étapes

- **Lister les grandeurs du modèle d'optimisation.** Toutes. Y compris celles que l'on croitISHED constantes : coûts, durées, capacités, taux de panne.

2. **Classer ces grandeurs par sensibilité de la solution.** On mesure cette sensibilité en perturbant chaque paramètre de dix pour cent et en observant le coût de la solution. C'est le classement le plus utile du projet.
3. **N'investir en apprentissage que sur les trois premiers.** Les autres sont soit stables, soit insensibles, et l'effort d'apprentissage sur un paramètre insensible est du travail purement perdu.

Ce classement prend une demi-journée et évite, dans la majorité des projets rencontrés, la majorité du travail d'apprentissage automatique.

Un modèle de prévision fournit une moyenne, l'optimiseur reçoit cette moyenne comme si elle était certaine, et produit un plan exactement optimal pour un scénario qui n'a pas eu lieu. L'erreur ne se manifeste jamais : le solveur a fait son travail. Elle n'apparaît qu'en exécution, sous forme d'un stock manquant, d'un camion qui attend, d'un client servi en retard. La parade est de passer la loi de probabilité au solveur, pas son espérance.

L'erreur classique : optimiser pour la moyenne

Astuce

Une prévision biaisée est pire qu'une prévision bruitée. Le bruit se moyenne sur les décisions successives ; le biais se cumulé et déplace durablement la solution. Pour cette raison, mesurer le biais d'un modèle de prévision est plus important que mesurer sa variance, et c'est le test que l'on oublié le plus souvent.

A retenir

- La RO optimise une décision, l'apprentissage estime les grandeurs dont elle dépend : la couture est le lieu de la valeur perdue.
- L'écart d'optimisation se borne par le produit de l'erreur d'estimation et de la sensibilité de la solution.
- Un paramètre mal estimé mais insensible ne coûte rien : classer par sensibilité avant d'investir.
- Optimiser pour la moyenne d'une prévision produit un plan exact pour un monde qui n'existe pas, sans jamais le signaler.
- Un biais se cumulé, un bruit se moyenne : mesurer lebiais avant la variance.

Chapitre 1.2

Prévision, décision et optimisation : la chaîne complète

La chaîne a quatre étages et une boucle de retour. Rompre la boucle est l'erreur la plus coûteuse.

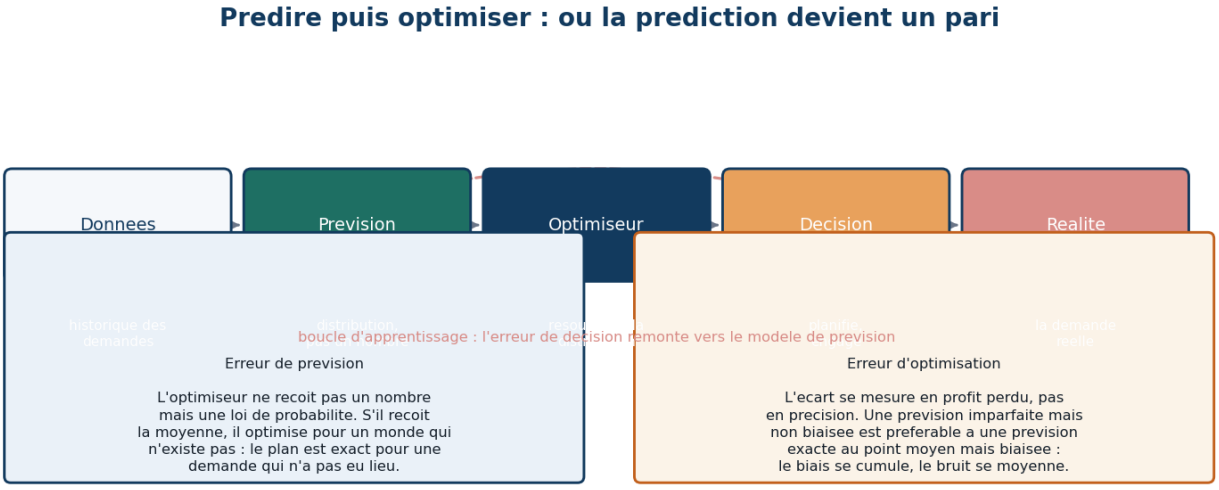


Figure 1 – Prédire puis optimiser : où la prédiction devient un pari

L'architecture canonique de l'hybridation comporte quatre étages : la donnée, la prévision, l'optimisation, la décision, puis un retour de la réalité qui réaligne le modèle de prévision. Chaque étage peut être excellent et le système complet être mauvais, parce que les critères de qualité des étages ne sont pas les mêmes.

$$x^\star_{\hat{c}} = \operatorname{argmin}_x \hat{c}^\top x, \quad \text{Régret} = c^\top x^\star_{\hat{c}} - c^\top x^\star_c \qquad \text{(R.2)}$$

Le regret est la seule métrique qui compte. Il compare le coût réellement payé au coût de la meilleure décision possible. Une erreur de prévision qui ne change pas la décision produit un regret nul, quelle que soit sa taille ; une erreur minime qui bascule la solution produit un regret important.

Étage	M métrique pertinente	Erreur typique	Conséquence de son ignorance
Donnée	couverture, fraîcheur, intégrité	valeurs manquantes silencieuses	le modèle apprend un biais de sélection
Prévision	biais par sous-groupe, calibration	erreur absolue moyenne faible mais biaisée	le regret croît linéairement
Optimisation	régret par rapport au plan exact	mesure de l'optimalité du modèle, pas du monde	on optimise le mauvais problème sans le savoir
Décision	coût économique de l'exécution	aucune mesure de la valeur perdue	aucune amélioration malgré un meilleur modèle

Tableau 1 – Ce qu'il faut mesurer à chaque étage

La ligne 2 est la plus contre-intuitive : la calibration vaut mieux que l'erreur absolue moyenne, parce qu'elle seule distingue un bruit d'un biais.

Mesurer la calibration, et non seulement l'erreur

Un modèle qui annonce 60 % de probabilité sur 100 cas doit en avoir environ 60 de bons. C'est la seule vérification qui distingue une prévision qui se trompe au hasard — donc inoffensive — d'une prévision qui se trompe systématiquement dans le même sens — donc coûteuse et silencieuse. La calibration se mesure par repliement : on regroupe les prédictions par tranche de probabilité, et on compare la fréquence observée à la probabilité annoncée. Un modèle bien calibré s'aligne sur la diagonale ; un modèle biaisé s'en écarte de façon systématique.

Algorithme 1 – Boucle d'alignement entre prédiction et exécution

```
Fixer la période d'évaluation et les groupes de segmentation
Calculer les prédictions sur la période, et conserver la loi, pas le point
Exécuter la décision issue de ces prédictions, et relever le coût réel
Comparer le coût réel au coût de la meilleure décision retrospectively possible
Décomposer le regret en part d'erreur de prévision et part de structure
Réentraîner sur les écarts, en pondérant davantage les groupes sous-représentés
Refaire le calcul : un même tour de comparaison ne prouve rien
```

Entrées : Historique de prédictions, décisions exécutées, coûts réels • **Sorties :** Régret par groupe et par période, modèle corrigé • **Complexité :** Coût dominant : levaluation, pas l'entraînement

La étape 5 est celle qui rend le travail utile. Un regret agrégé ne dit pas où chercher ; un regret décomposé par groupe et par type de décision désigne la correction.

Un modèle de prévision entraîné sur les conséquences de ses propres décisions passées ferme une boucle : il n'a jamais vu ce qui se serait produit sans sa recommandation, et il attribué à sa propre politique le mérite d'un monde qu'il a lui-même contribué à produire. La parade est de conserver un exploration minimum – une part des décisions prises au hasard ou par une règle fixe – dont la valeur contrefactuelle est observable. Cette part est le seul instrument qui permet d'estimer ce que le modèle ne ferait pas.

Réentraîner sur ses propres décisions

A retenir

- Le regret, et non l'erreur de modèle, est la seule métrique qui a du sens la valeur créée.
- La calibration distingue un bruit inoffensif d'un biais coûteux ; l'erreur absolue ne le fait pas.
- Décomposer le regret par groupe et par type de décision désigne la correction à faire.
- Entraîner sur les conséquences de ses propres décisions ferme la boucle et supprime toute contrefactualité.
- Une part d'exploration maintained est le seul moyen d'estimer ce que le modèle ne ferait pas.

Chapitre 1.3

Apprentissage par renforcement pour la décision séquentielle

La récompense est une fonction de conception. Un agent parfait sur une mauvaise récompense est un agent qui réussit le mauvais chose.

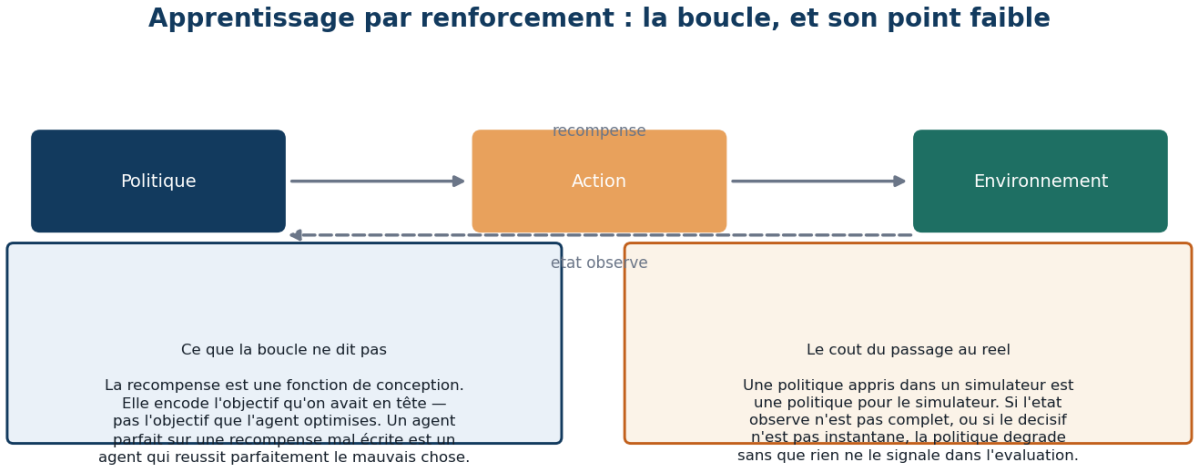


Figure 1 — Apprentissage par renforcement : la boucle, et son point faible

L'apprentissage par renforcement intervient lorsque la décision n'est pas un problème d'optimisation mais un enchaînement : le choix de maintenant modifie les options de plus tard. Un tournee d-optimalisolément n'est pas une politique bonne ; une politique bonne produit des décisions acceptables à chaque instant, y compris ceux où l'on ne regrettera pas l'optimum global.

$$\boldsymbol{\pi}^{\star} = \arg\max_{\boldsymbol{\pi}} \mathbb{E}_{\boldsymbol{\pi}} \left[\sum_{t=0}^{T} \gamma^{t} r(s_t, a_t) \right], \quad r = r_{\square 0 \square} - \lambda\, r_{\square 1 \square} \tag{R.3}$$

Les deux termes de la récompense ne sont pas du même ordre. Le premier est le revenu, mesurable et vérifiable. Le second est la pénalité de non-qualité, et son poids lambda est le paramètre le plus sensible de tout le système : il exprime la valeur d'une livre manquée, et une erreur d'un facteur dix sur lambda produit une politique qui optimise la mauvaise grandeur.

- **Récompense instantanée ou différée :** une récompense immédiate facilite l'apprentissage mais produit des agents qui sacrifient le long terme. Une récompense différée est la seule qui apprend un arbitrage correct, au prix d'une exploration beaucoup plus longue.
- **Masse de la récompense :** un facteur de décompte proche de un apprend des politiques myopes ; proche de zéro, il n'apprend rien, parce que l'horizon utile devient invisible.
- **Pénalité de risque :** ajouter un terme qui punit la variance des résultats plutôt que leur moyenne. C'est la seule façon d'obtenir une politique qui ne mise pas tout sur un scénario.
- **Exploration forcée :** sans exploration, la politique se concentre sur ce qu'elle connaît et devient figée. Une décroissance du taux d'exploration trop rapide produit un agent sous-optimal de façon irréversible.

Transférer une politique du simulateur au réel, en cinq contrôles

- **Bruiter l'observation pendant l'entraînement.** Un agent qui s'appuie sur un état parfait apprend une politique qui exige cet état parfait. Le bruit est le seul moyen de rendre la politique robuste à l'incertitude réelle.

2. **Introduire le retard de contrôle variable.** Un contrôleur réel réagit avec retard ; une politique entraînée sans retard oscille ou diverge dès que le mouvement s'accélère.
3. **Entraîner sur plusieurs horizons.** Un agent entraîné sur un problème de vingt décisions échoue sur cent, non par manque de capacité, mais parce que l'horizon n'a jamais été observé.
4. **Fixer la graine et publier l'écart-type.** Un résultat d'entraînement est un échantillon, pas une mesure ; trois graines suffisent à révéler une variance qui invalide une conclusion.

5. **Rejouer un scénario figé avant tout déploiement.** Un test de non-régression, choisi pour être représentant du pire cas réel, vaut mieux qu'une évaluation moyenne sur mille scénarios.

Ce que l'agent ne peut pas corriger seul

Un agent de renforcement apprend une politique cohérente avec sa récompense, pas une politique conforme aux règles de l'entreprise. Trois catégories de règles lui sont extérieures et doivent être appliquées après coup : les contraintes dures, qui rendent une décision impossible ; les contraintes de procédure, qui imposent un ordre que l'agent ignore ; et les interdictions, qui portent sur des actions que l'agent ne voit jamais dans ses données. Aucune quantité d'entraînement ne rendra un agent respecter une contrainte qu'il n'a pas observée violer.

Astuce

Une politique de renforcement est un actif de production difficilement réversible. Une règle d'ordonnancement se lit, se corrigé et se déploie en une heure. Avant de mettre un agent en production, garder la règle en service parallèle pendant une période complète de Representative, et comparer les deux décisions sur la même charge réelle.

A retenir

- La récompense encode l'objectif qu'on avait en tête, pas celui que l'agent optimise : la savoir écrire et la mesurer.
- Le poids de la pénalité de regret est le paramètre le plus sensible de tout le système.
- Le bruit d'observation, le retard et la variété des horizons sont les trois contrôles qui rendent le transfert défendable.
- Trois familles de règles restent extérieures à l'agent et doivent être appliquées après coup.
- Une politique apprise est difficilement réversible : garder la règle en service parallèle avant de basculer.

Chapitre 1.4

Logistique et tournées de livraison

Le problème de tournées n'a pas de solution exacte à l'échelle. Toute la question est de savoir laquelle on accepte.

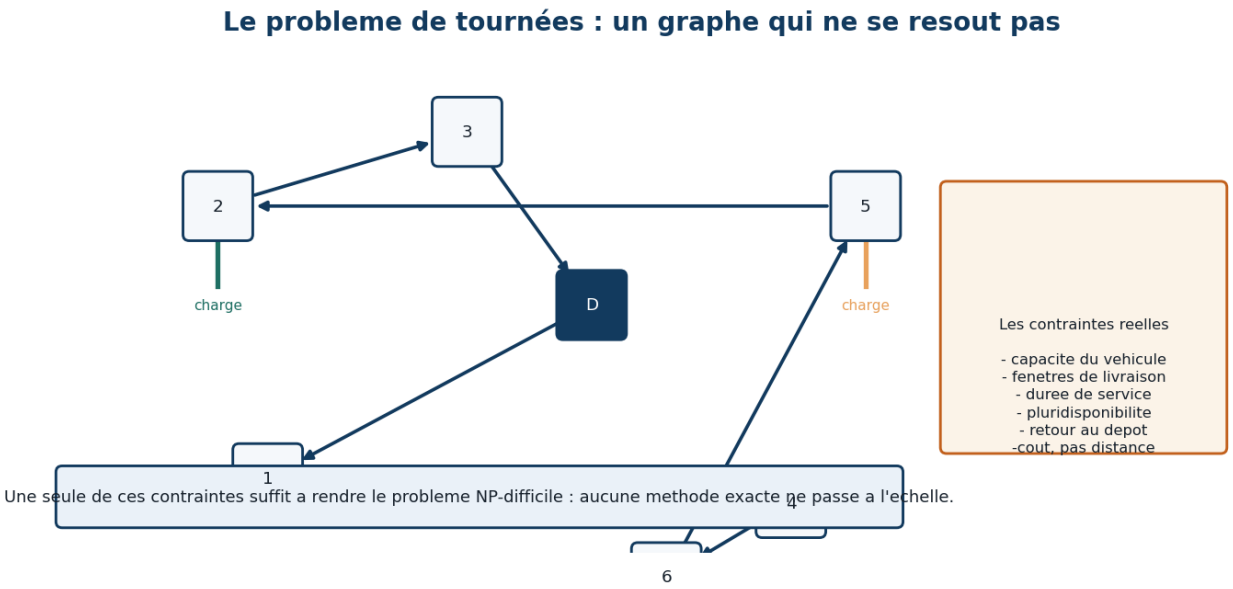


Figure 1 — Le problème de tournées : un graphe qui ne se résout pas

Le problème de tournées avec capacité et fenêtres horaires est l'un des rares problèmes de l'industrie dont on sait qu'il n'admet aucune solution exacte au-delà de quelques dizaines de points. Ce n'est pas une faiblesse des méthodes : c'est une propriété démontrée du problème. Le rôle de l'ingénieur n'est donc pas de chercher l'optimum, mais de produire une solution dont on sait dire à quel titre elle est acceptable.

$$\min \sum_{k=1}^{K} \sum_{i,j} c_{ij}\, x_{ijk} \quad \text{sous} \quad \sum_{k} \sum_{i} q_i\, y_{ik} \le Q_k,\ a_i \le \sum_{i \in V_k} \tau_i\, x_{i\ell\, k} \le b_i \tag{R.4}$$

Le coût est une somme d'arcs, la capacité est une contrainte de globale, et la fenêtre horaire est une contrainte de borne sur la somme des temps de service. Chacune suffit à rendre le problème NP-difficile : le Lagrangien relaxation résout relaxation, pas le problème.

Méthode	Ce qu'elle garantit	Budget typique	Usage
Exhaustive, branchement et coupe	l'optimum, pour petite taille	explose au-delà de 15 points	études de cas, références
Programmation linéaire	relaxation, jamais l'optimum	secondes à minutes	bornes inférieures, dimensionnement
Recherche locale (2-opt, Or-opt)	rien, mais elle monte vite	quelques secondes	qualité initiale
Métaheuristique guidée	rien, mais elle s'améliore	de quelques secondes à quelques minutes	le standard industriel
Apprentissage d'une politique	rien, et cela s'entraîne	entraînement long, exécution nulle	à grande échelle, après cadrage

Tableau 1 – Les solveurs et ce qu'ils garantissent

La dernière ligne est la seule qui change d'échelle. Elle ne remplace pas la métahéuristique : elle la remplace par une initialisation apprise, ce qui divisé le budget de recherche d'un ordre de grandeur.

```python
import math
from dataclasses import dataclass, field


@dataclass
class Commande:
"""Une demande a livrer, avec sa fenetre horaire et sa charge."""
identifiant: str
x: float
y: float
quantite: float
ouverture: float # debut de la fenetre, en minutes
fermeture: float # fin de la fenetre, en minutes
service: float = 8.0 # duree de service sur place

@property
def milieu(self) -> float:
return 0.5 * (self.ouverture + self.fermeture)


@dataclass
class Tournee:
"""Une solution partielle : un vehicule et son ordre de passage."""
vehicule: int
commandes: list = field(default_factory=list)
distance: float = 0.0
fin: float = 0.0

def cout_arcs(self, a, b) -> float:
return math.hypot(a.x - b.x, a.y - b.y)


```

Listing 1 — Insertion incrémentale et évaluation de faisabilité d'un plan de tournées

```python
def insertion_croissante(commande, tournees, capacite, depot, depart=0.0):
"""Insere la commande dans la tournee la moins dechiree.

Le choix se fait sur le cout marginal reel, et non sur une regle
du type « la tournee la plus proche ». Cette distinction explique
a elle seule la majorite de l'ecart entre une heuristique artisanale
et une methode structuree.
"""
meilleure = None
for t in tournees:
if t.vehicule == 0 and commande.identifiant != tournees[0].commandes[0].identifiant:
continue
charge = sum(c.quantite for c in t.commandes) + commande.quantite
if charge > capacite:
continue
points = [depot] + t.commandes
for pos in range(1, len(points) + 1):
avant = points[pos - 1]
apres = points[pos] if pos < len(points) else depot
gain = (t.cout_arcs(avant, apres)
- t.cout_arcs(avant, commande)
- t.cout_arcs(commande, apres))
if meilleure is None or gain < meilleure[0]:
meilleure = (gain, t, pos, avant, apres)
if meilleure is None:
return False
_, t, pos, avant, apres = meilleure
t.commandes.insert(pos - 1, commande)
t.distance += t.cout_arcs(avant, commande) + t.cout_arcs(commande, apres) \
- t.cout_arcs(avant, apres)
return True


```

```
def evaluer(tournees, commandes, depot):
"""Mesure la faisabilite, et non seulement le cout.

Une solution qui viole une fenetre horaire n'est pas une solution
moins bonne : c'est un plan inexecutable. Le retour doit donc
distinguer les deux, sinon la recherche convergera vers l'infaisable.
"""
index = {c.identifiant: c for c in commandes}
total, violations, retard = 0.0, 0, 0.0
for t in tournees:
t.fin = 0.0
precedent = depot
for c in t.commandes:
total += t.cout_arcs(precedent, c)
t.fin += t.cout_arcs(precedent, c) + c.service
if t.fin < c.ouverture:
t.fin = c.ouverture
if t.fin > c.fermeture:
violations += 1
retard += t.fin - c.fermeture
precedent = c
total += t.cout_arcs(precedent, depot)
return total, violations, retard
```

Calibrer un plan de tournées en cinq paramètres

- **Prix du temps d'attente.** Coût horaire du chauffeur, coût d'une promesse non tenue. C'est ce prix qui décide si une fenêtre horaire vaut la peine d'être respectée.

2. **Pénalité de violation, en cascade.** Un dépassement de cinq minutes ne coûte pas la même chose qu'un dépassement de deux heures. Une pénalité linéaire produit des plans absurdes aux extrémités.
3. **Durée de service réelle par point.** Estimée par catégorie de commerce, pas par défaut unique. C'est le paramètre le plus souvent sous-estimé, et il déplace tous les horaires.
4. **Temps de trajet par période.** La matrice de coûts n'est pas statique : le même trajet coûte trois fois plus aux heures de pointe. Une matrice unique produit un plan faux huit heures par jour.
5. **Part de commandes àHoraire souple.** En pratique, un dixième des clients acceptent une fenêtre élargie. Ce dixième vaut souvent plus que n'importe quelle amélioration d'algorithme.

Astuce

La matrice de coûts doit être reconstruite, pas seulement interpolée. Les temps de trajet réels par arc, appris sur les historiques de position, valent plus qu'une matrice théorique multipliée par un coefficient horaire global : le facteur d'encombrement n'affecte pas tous les arcs de la même façon.

Exercice 1.4.1 (difficulte : Moyen)

Un plan de tournées optimise sur une matrice de temps statiques réduit la distance parcourue de 12 %. En exécution, le temps de conduite réel est supérieur de 19 % à celui prévu, et le taux de livraisons hors fenêtre passe de 4 % à 17 %. Proposer trois corrections, classées par rapport effort / bénéfice attendu.

Indice : Le gain théorique et le gain réel ne sont pas le même chiffre : l'écart vient de la matrice, pas du solveur.

Correction

- 1. Matrice de coûts horaire, reconstruite à partir des temps réellement constatés par arc et par tranche horaire. C'est la correction de loin la plus rentable, et elle ne demande aucun changement d'algorithme. 2. Pénalité de dépassement en cascade plutôt que linéaire, qui rend le solveur honnête sur les cas extrêmes. 3. Réduction du taux de hors-fenêtre par renégociation commerciale — dix pour cent des clients élargissent leur fenêtre — ce qui coûte rien en temps de calcul et rend le problème réellement plus facile. Le solveur, lui, n'est en aucun cas le problème.

A retenir

- Le problème de tournées n'a pas de solution exacte au-delà de quelques dizaines de points : c'est une propriété démontrée, pas une faiblesse de méthode.
- Une solution qui viole une fenêtre horaire n'est pas une solution moins bonne, c'est un plan inexécutable.
- La matrice de coûts horaire, reconstruite sur des temps réellement constatés, corrige plus de gain que tout algorithme.
- Le prix du temps d'attente est le paramètre qui décide si une contrainte horaire vaut la peine d'être respectée.

Chapitre 1.5

Ordonnancement et pilotage d'atelier

Un atelier est un graphe. Le dire change ce qu'on peut apprendre, et ce qu'on peut garantir.

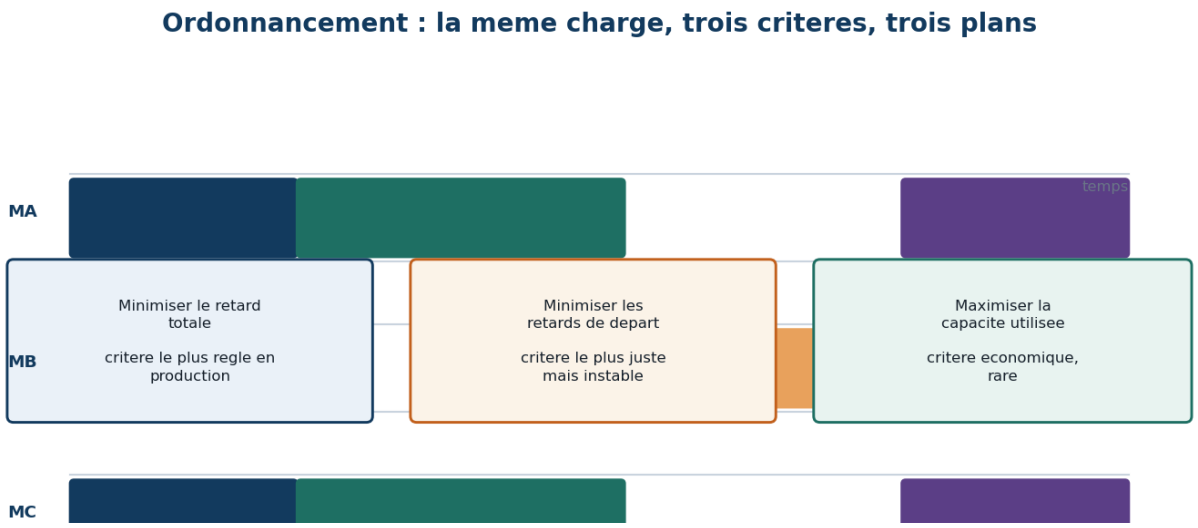


Figure 1 – Ordonnancement : la même charge, trois critères, trois plans

Un atelier de fabrication se décrit naturellement comme un graphe : les nœuds sont les opérations et les machines, les arêtes sont les contraintes de précédence d'une part et de partage de ressource d'autre part. Cette description n'est pas une commodité : elle détermine ce qu'un modèle d'apprentissage peut apprendre, et ce qu'un solveur peut garantir.

$$C = \max_{j} \leq ft\ C_{j-1},\ \max_{k \in \square 0 \square (j)} \leq ft(\ S_{kj} + C_{k}\ ),\ R_{j} = d_{j} - C_{j} \qquad (R.5)$$

La première expression est la règle de Wheeler : la complétion d'une opération est le maximum entre son début au plus tôt et la fin de toutes ses antécédentes. C'est l'algorithme glouton exact pour un atelier à une machine par station, et il est faux dès qu'une opération dispose de plusieurs machines. La seconde est le retard, qui est la grandeur qu'on optimise réellement.

Critère	Formulation	Plan obtenu	Limite
Temps de cycle total	minimiser le makespan	peu d'attente, lots regroupés	ignore totalement les promesses
Retard total	minimiser $\sum_j R_j$	les commandes urgentes passent d'abord	peut sacrifier une commande pour en sauver dix
Retard maximal	minimiser $\max_j R_j$	aucune commande n'est abandonnée	plus instable, sensible à l'ordre d'entrée
Capacité utilisée	maximiser le taux d'occupation	machines saturées en permanence	rare : aucune machine n'est jamais à 100 % utile

Tableau 1 – Trois critères d'ordonnancement et ce qu'ils produisent

Trois critères, trois plans différents, pour la même charge. Choisir le critère sans demander au métier est la cause la plus fréquente de mécontentement sur un ordonnanceur.

- **Le graphe de contraintes est un objet de domaine, pas une illustration.** Les nœuds sont les opérations ; les arêtes de précédence sont des invariants durs ; les arêtes de partage sont des relations de ressource, qui changent avec la disponibilité.
- **Les attributs de nœud sont ce qui rend le graphe apprenable.** Temps de traitement, attente cumulée, échéance, et disponibilité de la ressource. Un graphe sans attributs n'apprend que sa topologie, qui ne change pas d'un tour à l'autre.
- **La propagation est la seule information réellement nouvelle.** Quand une machine ralentit, ce qui compte n'est pas le graphe lui-même, mais la façon dont l'information se propage le long des arêtes de contention. C'est exactement ce que fait un réseau de messages sur graphe.
- **La contrainte reste vérifiée par la RO, jamais par l'apprentissage.** Le modèle propose un ordre ; un contrôleur par contraintes vérifie qu'il est réalisable. Confier cette vérification au modèle, c'est lui demander de garantir ce qu'il n'a pas été conçu pour garantir.

Cadrer un système d'ordonnancement apprenant, en six points

- **Décrire l'atelier comme un graphe** et figer ce graphe dans un test. Toute évolution ultérieure doit passer ce test.

2. **Séparer les invariants des paramètres.** Une précédence est un invariant ; une durée de traitement est un paramètre. Les mélanger dans un même vecteur d'état empêche tout apprentissage.
3. **Choisir la règle de repli avant l'agent.** Quelle règle s'applique si le modèle est indisponible ? Cette question doit avoir une réponse écrite, pas une intuition.
4. **Fixer la graine et publier l'écart-type.** Un gain annoncé sans intervalle n'est pas un résultat.
5. **Définir la mesure de valeur en euros par heure.** Sans conversion monétaire, aucun arbitrage n'est possible entre un gain de 2 % de temps de cycle et une commande sauvée.
6. **Prévoir la journalisation de chaque décision.** C'est elle qui permettra, dans six mois, de comprendre pourquoi le performance s'est dégradée après un changement de machine.

Ce que l'apprentissage par graphe n'apporte pas

Un réseau de messages sur graphe apprend la propagation de l'information dans un atelier. Il n'apprend pas la physique de la machine, pas la disponibilité future d'un opérateur, et pas la durée réelle d'un réglage. Sur un atelier où les pannes couvrent la moitié du temps, la qualité de la politique est bornée par la qualité de la description des pannes — et l'apprentissage n'y change rien.

A retenir

- Un atelier est un graphe : les nœuds sont les opérations, les arêtes sont les précédences et les contentions.
- Trois critères d'ordonnancement produisent trois plans différents pour la même charge : le choisir est un acte de gestion.
- La règle de Wheeler est exacte à une machine par station, et fausse dès qu'une opération a le choix.
- Apprendre la propagation de l'information ne dispensé pas de décrire correctement le domaine.

- La vérification des contraintes reste à la programmation par contraintes, jamais au modèle.

Chapitre 1.6

Énergie, finance et déploiement en production

Un système hybride en production a deux dérives : celle des données, et celle du modèle. Elles se mesurent différemment.

Les deux secteurs les plus contraintes chimiquement imposent la même discipline : la prévision de production et de consommation pour l'énergie, l'espérance de rendement et la matrice de covariance pour la finance. Dans les deux cas, la décision d'optimisation est un arbitrage permanent entre rendement et risque, et l'apprentissage n'intervient que sur l'estimation de ces deux grandeurs.

	Réseaux électriques	Portefeuilles financiers
Incertitude dominante	production renouvelable	évolution des cours
Contrainte dure	équilibre offre et demande, en continu	interdits de vente à découvert, liquidité
Structure de décision	intégration sur horizon glissant	allocation fractionnaire sous contraintes
Horizon d'engagement	minutes à un jour	jours à plusieurs années
Coût d'une erreur	défaillance généralisée, en cascade, en cascade	insolvabilité, perte de confiance
Ce que l'IA apporte	prévision à court terme, courtage	espérance de rendement, corrélations
Ce que la RO apporte	respect de l'équilibre, faisabilité	respect des contraintes, liquidité

Tableau 1 – Énergie et finance : mêmes principes, contraintes différentes

La ligne des contraintes dures est la plus instructive. Dans les deux cas, l'optimiseur ne peut pas produire une solution qui viole l'équilibre électrique ou l'interdit de vente, quel que soit le rendement annoncé.

$$\max_{\mathbf{w}} \boldsymbol{\mu}^{\top}\mathbf{w} - \frac{\lambda}{2}\mathbf{w}^{\top}\boldsymbol{\Sigma}\mathbf{w}, \quad \mathbf{w}^{\top}\mathbf{1} = 1, \mathbf{w} \geq 0 \tag{R.6}$$

La matrice de covariance est le paramètre estimé, et c'est le seul. Une erreur sur la moyenne de rendement produit un écart faible ; une erreur sur la covariance produit un portefeuille diversified autrement, et l'erreur est systématique car elle se reproduit à chaque réallocation.

- **Séparer la couche prédictive de la couche d'optimisation.** La première est rapide et sans état ; la seconde est lente et coûteuse. Les exécuter dans le même processus rend le premier indisponible quand le second est lent.
- **Pré-calculer les scénarios.** Les scénarios de demande ne sont pas des inconnues à chaque appel : ce sont des scénarios. Les pré-calculer, et ne conserver en mémoire que ceux qui survivent à un filtre de faisabilité, divisé le temps de réponse d'un ordre de grandeur.
- **Règle de repli explicite.** Un algorithme glouton et un ordonnancement statique sont disponibles. Les écrire avant d'en avoir besoin est la seule façon qu'ils fonctionnent le jour où le modèle tombe.
- **Surveiller deux dérives séparément.** La dérive des données change ce que le modèle prédit ; la dérive du modèle change ce que le solveur considère comme possible. Elles ont des signatures différentes et des causes disjointes.

Rendre un système hybride défendable, cinq contrôles

- **Régret mesuré en euros, par période et par segment.** C'est le seul chiffré qui rapproché le travail technique de la décision d'investissement.

2. **Écart-type sur trois graines minimum.** Une amélioration inférieure à l'écart-type n'est pas une amélioration.
3. **Test hors échantillon sur une période de production réelle.** Pas sur un jeu de validation, jamais.
4. **Règle de repli testée en panne simulée.** Une règle de repli jamais exercée est une règle de repli inexistante.
5. **Journal de décision exportable.** Qui, quand, quelle recommandation, quelle exécution, quel coût constaté. Sans ce journal, aucun audit n'est possible et aucune amélioration n'est attribuable.

Cadre juridique de l'aide à la décision automatisée

Un système qui recommande une allocation financière, un ordonnancement ou un approvisionnement assisted-conserve plusieurs obligations. Les recommendations destinées à des clients doivent être identifiables comme telles. Les décisions à effet juridique ou financier engagent une obligation d'information et d'explicabilité, et relèvent de la réglementation sectorielle applicable. La conservation des journaux de recommandation et d'exécution est une condition de conformité, pas une fonctionnalité optionnelle. Un système hybride n'échappe à aucune de ces obligations par le fait qu'il soit semi-automatique : la part laissée à l'humain ne change rien à la responsabilité de l'exploitant.

Astuce

La métrique d'un système hybride n'est jamais la qualité de son modèle. C'est l'écart entre le coût de la meilleure décision possible et le coût réellement payé, par segment. Cette seule mesure permet de décider si le prochain jour d'effort produit de la valeur.

Projet – Concevoir et cadrer un module de décision hybride

Duree : 6 semaines | Equipe : 3 a 5 etudiants | Depot : gitlab + rapport

Une entreprise de distribution souhaité ses coûts de kilomètre de 12 % en six mois, sans recruter. Elle dispose d'un an d'historique de tournées, d'un service web interne, et d'une équipe de trois développeurs.

Missions

1. Décrire le problème d'optimisation : variables de décision, contraintes dures, contraintes souples, et critère de coût.
2. Lister les grandeurs du modèle et les classer par sensibilité de la solution, en justifiant le classement par une mesure.
3. Formuler la politique de prévision retenue, en indiquant ce qu'elle estime et ce qu'elle délègue à l'optimiseur.
4. Écrire la règle de repli, et décrire le test qui démontrera qu'elle fonctionne en panne simulée du modèle.
5. Définir la métrique de valeur et le protocole de mesure : regret, période, segments, nombre de graines.
6. Écrire le plan de mise en service parallèle : durée, critère de bascule, et conduite en cas d'écart défavorable.

Livrables

- N
- o
- t
- e
- d
- e

- c
- a
- d
- r
- a
- g
- e
- e
- n
- d
- i
- x
- p
- a
- g
- e
- s
- ,
- t
- a
- b
- l
- e
- a
- u
- d
- e
- s
- e
- n
- s
- i
- b
- i
- l
- i
- t
- é

- d
- e
- s
- p
- a
- r
- a
- m
- è
- t
- r
- e
- s
- ,
- p
- r
- o
- t
- o
- c
- o
- l
- e
- d
- e
- m
- e
- s
- u
- r
- e
- d
- u
- r
- e
- g
- r
- e
- t

- ,
- r
- è
- g
- l
- e
- d
- e
- r
- e
- p
- l
- i
- t
- e
- s
- t
- é
- e
- ,
- p
- l
- a
- n
- d
- e
- b
- a
- s
- c
- u
- l
- e
- .

Évaluation

- L
- e

- j
- u
- r
- y
- v
- é
- r
- i
- f
- i
- e
- t
- r
- o
- i
- s
- p
- o
- i
- n
- t
- s
- :
- l
- e
- c
- l
- a
- s
- s
- e
- m
- e
- n
- t
- d
- e
- s

- e
- n
- s
- i
- b
- i
- l
- i
- t
- é
- e
- s
- t
- j
- u
- s
- t
- i
- f
- i
- é
- p
- a
- r
- u
- n
- e
- m
- e
- s
- u
- r
- e
- ,
- l
- a
- r
- è

- g
- l
- e
- d
- e
- r
- e
- p
- l
- i
- a
- é
- t
- é
- e
- x
- e
- r
- c
- é
- e
- ,
- e
- t
- l
- e
- p
- r
- o
- t
- o
- c
- o
- l
- e
- a
- n
- n

- o
- n
- c
- e
- u
- n
- i
- n
- t
- e
- r
- v
- a
- l
- l
- e
- d
- e
- c
- o
- n
- f
- i
- a
- n
- c
- e
- p
- l
- u
- t
- ô
- t
- q
- u
- '
- u
- n

- g
- a
- i
- n
- p
- o
- n
- c
- t
- u
- e
- l
- .

A retenir

- Un système hybride a deux dérives distinctes, aux signatures disjointes : la dérive des données et la dérive du modèle.
- La covariance et la production renouvelable sont les paramètres les plus critiques, parce que leur erreur est systématique.
- Une règle de repli jamais exercée est une règle de repli inexistante : il faut la tester en panne simulée.
- La métrique d'un système hybride est le regret, mesuré en valeur économique et par segment.
- La semi-automatisation ne réduit aucune obligation de conformité ni de traçabilité.

Abréviations, notation et ressources de consultation

Annexes

Annexes

Ce cours se termine par deux annexes de référence. La première donne la signification de chaque sigle employé, sous les deux formes — sigle et développé — car la litterature emploie indifféremment l'une ou l'autre. La seconde donne la signification de chaque symbole mathématique : elle permet de relire une démonstration sans revenir au chapitre dont elle est extraite, et de vérifier qu'un même symbole garde le même sens d'un chapitre à l'autre.

Chapitre A.1

Liste des abréviations

Les sigles ci-dessous sont ceux que le cours emploie. Chacun est donné avec sa forme développée et le sens précis qu'il prend ici, qui n'est pas toujours celui qu'on lui donne ailleurs.

Un sigle employé sans être développé oblige le lecteur à le reconnaître au passage, ou à l'abandonner. Le tableau ci-dessous ferme la liste de ceux que le cours emploie : il est exhaustif pour ce document, et non pour la discipline.

Sigle	Forme développée	Sens dans ce cours
GAP	Écart de relaxation	Écart relatif entre la borne inférieure et la solution entière.
LP	Linear Programming	Programmation linéaire : optimisation d'un critère linéaire sous contraintes linéaires.
MILP	Mixed-Integer Linear Programming	Programmation linéaire à variables entières et continues.
NP	Non-Polynomial time	Classe de complexité des problèmes dont on ignore l'appartenance à P.
PDDL	Planning Domain Definition Language	Langage de description de problèmes de planification.
RO	Recherche opérationnelle	Optimisation de décisions sous contraintes.
SPFA	Shortest Path Faster Algorithm	Algorithme de plus court chemin fondé sur une file d'attente.
TCD	Taux de couverture de la demande	Part des demandes servies dans le délai imparti.
VRP	Vehicle Routing Problem	Tournées d'une flotte de véhicules qui livrent une demande dispersée.

Tableau 1 – Abréviations employées dans le cours

Chapitre A.2

Table de notation

Les symboles ci-dessous gardent le même sens d'un chapitre à l'autre. Lorsqu'un chapitre doit réemployer un symbole pour un autre objet, il le redéclare sur la page même.

La notation est celle du cours : vecteurs en gras, matrices en majuscules grasses, scalaires en italique. Les espérances sont notées $\mathbb{E}$, les lois de données $\mathcal{D}$, les espaces de paramètres Θ.

Symbole	Signification
x_j	Quantité de la variable de décision j.
c_j	Coefficient de coût unitaire de la variable j.
a_{ij}	Coefficient de la contrainte i pour la variable j.
b_i	Second membre de la contrainte i.
m	Nombre de contraintes.
n	Nombre de variables de décision.
$c^\top x$	Critère à minimiser ou à maximiser.
$Ax \leq b$	Ensemble des contraintes linéaires.
$x \in \mathbb{Z}_+^n$	Variables entières positives ou nulles.
u_i	Variable d'écart de la contrainte i.
z^*	Valeur optimale du critère.
LP	Valeur optimale de la relaxation linéaire.
IP	Valeur optimale du problème entier.
GAP = (LP - IP)/(IP)	Écart relatif de relaxation.
p_i	Probabilité de réalisation du scénario i.
d_{ij}	Distance entre les sites i et j.
T	Durée de l'horizon de planification.

Tableau 1 – Symboles mathématiques et leur signification

> **Convention**
> Toute divergence constatée entre cette table et un chapitre signale une erreur de transcription, non une définition nouvelle : la table fait foi.

> **A retenir**
> - Un sigle employé sans être listé ici est un oubli de l'auteur, non une définition nouvelle.
> - Un symbole qui change de sens d'un chapitre à l'autre rend toute démonstration ininterprétable : c'est la principale cause des contresens que nous avons relevés.
> - Ces deux tables sont les seules parties du cours à maintenir à la main : le reste se relit tout seul.

Index des figures et des tableaux

Référence rapide du matériel visuel

Figures

1. Deux compétences complémentaires, et la couture entre les deux
1. Prédire puis optimiser : où la prédiction devient un pari
1. Apprentissage par renforcement : la boucle, et son point faible
1. Le problème de tournées : un graphe qui ne se résout pas
1. Ordonnancement : la même charge, trois critères, trois plans

Tableaux

1. Ce que chacune des deux disciplines sait faire et ignorant
1. Ce qu'il faut mesurer à chaque étage
1. Les solveurs et ce qu'ils garantissent
1. Trois critères d'ordonnancement et ce qu'ils produisent
1. Énergie et finance : mêmes principes, contraintes différentes
1. Abréviations employées dans le cours
1. Symboles mathématiques et leur signification

Fin du document

Intelligence Artificielle Avancée & Federated Learning — Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Ghiloufi Jamel, Architecte Logiciel Principal — Pixel Software Design
Département de Recherche & Développement — Recherche
Pixel Software Design — Édition 2026

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Document généré automatiquement le 29/09/2026 – mise en page Pixel Software Design.